

Detecting disguised processes using Application-Behavior Profiling

Christian Vaas and Jassim Happa

Department of Computer Science, University of Oxford, UK
firstname.surname@cs.ox.ac.uk

Abstract—In order to avoid detection, malware can disguise itself as a legitimate program or hijack system processes to reach its goals. Commonly used signature-based Intrusion Detection Systems (IDS) struggle to distinguish between these processes and are thus only of limited use to detect such attacks. They also have the shortcoming that they need to be updated frequently to possess the latest malware definitions. This makes them inherently prone to missing novel attack techniques. Misuse detection IDSs however overcome this problem by maintaining a ground truth of normal application behavior while reporting deviations as anomalies. In our approach, we try to accomplish this by observing a process’ memory consumption. This is for two reasons: We expect the readings to be less volatile in comparison to for instance network operations. Second, by breaking the problem down, we are able to investigate thoroughly while still laying the foundations for future expansion. We use the observations from a given host to train a machine learning algorithm. After an initial learning phase, we evaluate the model with readings from the application it has been trained on and other applications in order to assess its quality. Our results indicate that the efficacy of this method is highly dependent on parametrizing the machine learning algorithm appropriately. A large variance in accuracy with only slightly altered inputs confirms this suggestion. We finish with a discussion on deploying such an IDS at scale in a realistic scenario.

I. INTRODUCTION

There are many ways malware can avoid detection. One such way is to disguise itself as a legitimate program or to hijack system processes to reach its goals. This can start with the visit of an infected website that exploits a vulnerability in a browser plug-in [1], [2] and ranges over legitimate downloads infected via a man-in-the-middle attack [3]. A common technique to prevent plug-in based intrusion is sandboxing of browser tabs [4]. Anti-virus software widely uses signature based detection [5] to recognize infected executables among incoming files. And several approaches to profile user activities and detect deviation from *normal* behavior have been established under the topic of continuous authentication [6], [7], [8] as well as insider threat detection [9], [10].

A common detection evasion technique for malware is to disguise itself as another process, either by hiding inside the application’s binary or through direct impersonation. In this paper, we propose a novel approach to identify anomalous behavior of a machine’s processes on a per-application basis. This covers an attack where malware disguises itself as a legitimate application but its processes expose unusual activity. On Windows machines *svchost.exe* can be used as a facade to attach malicious code or to name processes identically in order to avoid detection [11]. Our method uncovers such mole processes, independent from the used attack vector,

through behavior learning and continuous verification. Thus, an examination of unusual behavior reduces the chances for malware to circumvent our detection mechanism.

We will use the terms *application* and *process* throughout this paper as follows. We define a process to be an instance of an application. This means that even though two processes are created from the same application executable, they might differ in their behavior as they could be used to complete different tasks. Further, we characterize a process’ behavior by its virtual memory consumption. For our case study, we implemented a data-acquisition client which gathers snapshots of all processes on a machine over a given period of time. The record for each process includes its resource utilization, used as a fingerprint for that process, and a timestamp. According to our hypothesis, these n -dimensional data points form clusters that can be used in machine learning algorithms to establish a model for each application. After an initial learning phase, we present anomaly detection results based on those models. Finally, we discuss our results and outline future directions for this type of anomaly detection while considering scalability.

II. RELATED WORK

The literature on detection systems can be divided into two categories: (a) Misuse detection, which requires the system to be aware of malicious behavior patterns prior to the actual attack and (b) anomaly detection. The latter establishes a behavior baseline of a system at run-time and then monitors it for deviations [12]. While machine-learning techniques can be very effective at anomaly detection [13] and allow for the detection of novel attacks, this approach is also prone to trigger false alarms. The main cause of this is behavior not accounted for during model generation. Hence, the quality of these algorithms needs to be assessed to measure their merit. This is often done using the False Positive Rate (FPR) [14], denoting the system recognizing normal behavior as anomaly, and the False Negative Rate (FNR) denoting the fraction of malicious activity not detected as such.

Previous approaches for intrusion detection used program system calls as features for behavior classification. Sato *et al.* [15] generated models for applications by recording system calls associated with a particular process which are then ranked by frequency. The resulting histogram is compared to an example run to quantify divergence to the model. Hofmeyr *et al.* [16], [17] also based their approach on system calls for intrusion detection. Moreover, they investigated the feasibility of entire call sequences to uncover adversarial activities.

Ryan *et al.* [18] on the other hand use a neural network with back propagation for intrusion detection. They identified

a set of basic operations as features for a user's behavior. After recording the execution of these commands on a machine for ten users over the course of twelve days. This data was then used to train a neural network. Their results indicated that such a system can be used to identify individual users merely based on the programs they run. However, the approach did not turn out to be suitable for real-time detection since a long training phase is required for a neural network to produce reasonable results.

III. METHOD

Our methodology is structured into three phases.

- **Acquisition phase:** gathering the digital fingerprint of a target machine's processes.
- **Learning phase:** deriving a model for each application and using cross-validation to refine the model's parameters.
- **Production phase:** assessing the quality of the model using standard machine learning methods.

Our approach characterizes a process by its virtual memory utilization for two reasons. Firstly, we expect the memory utilization to be less volatile in comparison to network operations or CPU usage readings. Secondly, we use virtual rather than physical memory consumption, as the latter does not account for the amount of memory swapped to the hard drive. This mitigates the risk of a malicious process allocating a large amount of memory in order to force other process' data into the swap partition and thus excluding it from our measurements.

A. Acquisition phase

We gather the memory fingerprint of a target machine using *psutil* [19]. It allows us to record snapshots of all processes including their resource consumption. The result is a single time series of memory usage for each process as shown in Figure 1. Due to the execution time of *psutil*, the acquisition is limited to a sampling interval of 60 seconds. Further, a snapshot may include multiple processes of the same application which we incorporate into our model as individual runs.

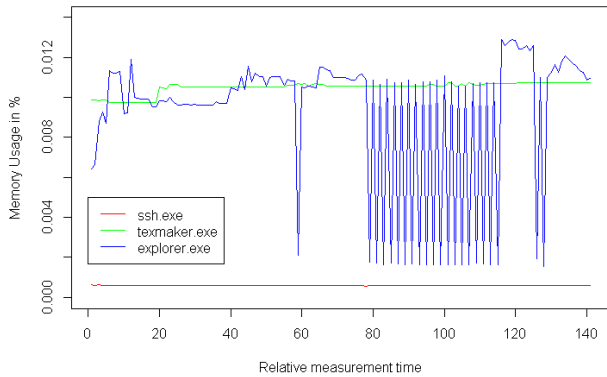


Fig. 1: Time series of virtual memory for three processes.

Learning phase: In this phase the algorithm computes a model for every application. It consists of a **fingerprint**, a **threshold** and a **threshold factor** which are explained in detail throughout this section. The features we use for the model

are derived from the linear data stream of the time series aggregated using a sliding window, as shown in Figure 2 for a window size of four. The data points of the resulting data sequence each consist of a four-dimensional vector. The goal of the window is to capture patterns within the stream [20]. The larger the window, the more data is considered part of the pattern. Moreover, the windowing approach is time-independent as it only preserves the order for the values within but not among individual windows. Figure 3 shows the fingerprints of six applications plotted in the feature space. Each color encodes a different applications, i.e. all orange dots belong to the same application and so forth. The window size chosen for this diagram is 25, which means each point represents a 25-dimensional vector. As we cannot represent 25 dimensions in the Cartesian plane, we use the t-SNE visualization method proposed by Maaten and Hinton [21]. It projects a n -dimensional data set into two-dimensional space with some moderate loss in precision.

Besides the fingerprint of an application, we also need to determine a threshold as well as a threshold factor that will later enable anomaly detection. Listing 1 shows the model generation process. The algorithm draws an instance *window* from the windowed data. For this, it computes the Euclidean distance to the closest point in the current fingerprint. Figure 4 displays the first three steps of the training phase. The gray area visualizes the threshold value. Initially, it is set to zero, thus the entire area is white. After the second point has been added, the algorithm computes distance a between the two points to determine the new threshold. The third point increases the threshold to b which is used to update the radius around all points. Intuitively, we determine the shortest distance between the new point and the ones already included in the fingerprint. After this, we set the threshold to the maximum of these distances as we still operate under the assumption that the observed behavior is benign and thus part of the ground truth which should be included into the fingerprint. In case of the minimum distance being equal to zero we discard the point as a duplicate which is already represented in the model. Points outside the gray area will later be considered outliers and thus anomalies.

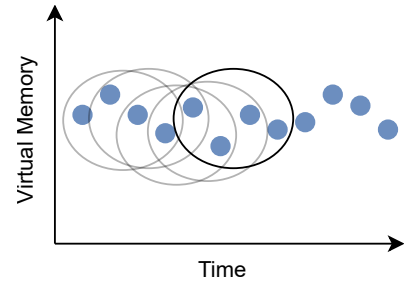


Fig. 2: A sliding window of the size four capturing patterns from the time series.

The generated model can be used as a binary classifier to categorize future data points into normal and anomalous behavior. Listing 2 shows the classification algorithm. For each new windowed reading, we iterate over the fingerprint of a model to determine the distance to each item. If the distance is zero, the encountered data point is identical to one in the fingerprint, and the verification can be terminated. Otherwise, we check whether the distance exceeds the threshold modified

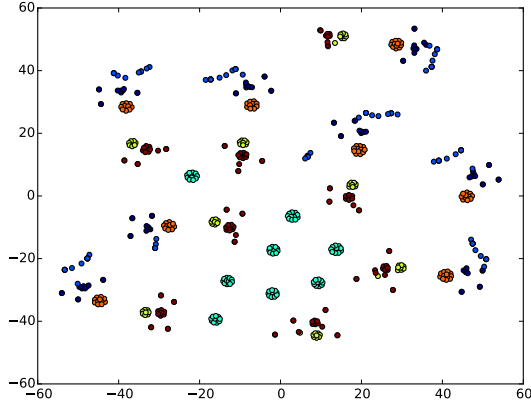


Fig. 3: Features of six processes with a window size of 25. Each color represents a different application.

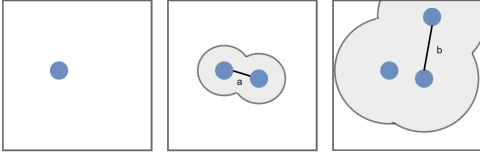


Fig. 4: The first three steps in the training phase, visualizing the threshold values. Initially the threshold is zero (left); with the second data point it expands to *a* (middle); with the third point it expands to *b* (right)

by a multiplicative constant, *factor*. If so, we increase a counter which is maintained on a per-application basis. This counter serves as buffer and is regularly checked by a sentinel process which generates an alarm event once a given threshold is exceeded.

```

for window in windowedData:
    #window = [0.4, 0.3, 0.5, 0.3]
    minimum = float('infinity')
    for inst in fingerprint:
        dist = np.linalg.norm(window - inst)
        minimum = min(dist, minimum)
    if minimum == 0:
        continue
    if len(fingerprint) > 0:
        threshold = max(threshold, minimum)
    fingerprint.append(window)

```

Listing 1: Model generation from windowed data.

```

dist = float('infinity')
for inst in fingerprint:
    d = np.linalg.norm(window - inst)
    dist = min(dist, d)
    if d == 0.0:
        break
if dist > factor * threshold:
    anomalies += 1

```

Listing 2: Binary classifier to distinguish anomalous from normal behavior.

Using this model generation and anomaly detection algorithm we can evaluate the performance of each model and determine its ideal parameters: *window size* and *threshold factor*.

An established accuracy measure for binary classification is a model's F_1 score. It takes the True Positive Rate (TPR), FNR and FPR into account to calculate an average of precision and recall by weighing those values accordingly, as shown below in Equation 1. We can thus use this score as a proxy to maximize the performance of a model.

$$F_1 = \frac{2 \times TPR}{2 \times TPR + FNR + FPR} \quad (1)$$

IV. RESULTS

To test our algorithm, we acquire process data of a user's machine over the course of one day. Afterwards, the data is divided into two groups making up for 5/6 and 1/6 of the entire data, respectively. We will call the former the *training set*, which is used during the training phase, and the latter *evaluation set*, used to simulate the production phase. In a second step, we create five evenly sized partitions from the training set. The resulting pieces of data are shown in Figure 5.

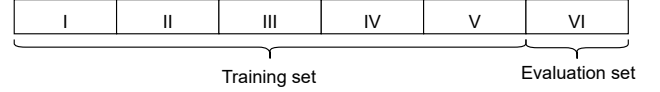


Fig. 5: Partitioning of data into training and evaluation set. The training set is used for a 5-fold-cross-validation.

This allows us to perform a 5-fold-cross-validation as described by Zhang [22]. The goal of this is to prevent our model from being over-fitted. First, we choose an application we want to train on. We decided to use *explorer.exe* due to its omnipresence on Windows machines. Afterwards, we select every possible four-permutation of the five blocks, train a model with it and evaluate its performance against the fifth portion combined with the training data of all other processes. Table I shows the results of a cross-validation run from our data set for a window size of 28 and a threshold factor of two. The *total* column lists the amount of memory utilization readings used to validate the respective model, while the *anomalies* column shows the anomalies among those. For the first run, which uses Set I for validation, two out of the 61 instances were labeled as anomalies. Furthermore, 4871 out of the 5000 measurements from other processes have been correctly recognized as not taken from *explorer.exe*. For the cross-validation with the Sets II-V we can report similar good performance.

The overall performance across all five cross-validations is shown in Table II. The mean and standard deviation are recorded for FPR and TPR in the first and second rows respectively. In our particular scenario the true positives are the processes that are not *explorer.exe* and recognized as such, i.e. anomalies. False negatives are the instances that are recognized as *explorer.exe* but are actually from a different process, and false positives are the readings from *explorer.exe* that are classified as anomalies. We can observe a high TPR and a low FPR which indicates that the algorithm performs well within the boundaries of the training set. A high TPR is desirable since it means that the majority of malicious behavior is detected while a low FPR improves the usability and hence the acceptance by the users.

Set I for validation - threshold: 0.0017480		
	Anomalies	Total
explorer.exe	2	61
!explorer.exe	4871	5000
Set II for validation - threshold: 0.003677		
	Anomalies	Total
explorer.exe	0	60
!explorer.exe	4868	5000
Set III for validation - threshold: 0.003677		
	Anomalies	Total
explorer.exe	0	60
!explorer.exe	4868	5000
Set IV for validation - threshold: 0.003677		
	Anomalies	Total
explorer.exe	0	60
!explorer.exe	4866	5000
Set V for validation - threshold: 0.003677		
	Anomalies	Total
explorer.exe	0	60
!explorer.exe	4856	5000

TABLE I: Cross-validation performance for *explorer.exe*.

	mean	std
explorer.exe	0.032787	± 0.065574
!explorer.exe	0.975880	± 0.004019

TABLE II: Aggregated performance over the five cross-validation runs.

These values are now computed for several combinations of *threshold factor* and *window size* to determine the most representative model [23]. Therefore, we iterate over an initial range of window sizes from two to fifty. This range was chosen since a small window size captures almost no behavior while a large window size reduces the significance of each anomalous observation over the window, thus decreasing the effectiveness of the detection. And two, windows with a large size, such as fifty, are too large for single observations to be significant enough to cause a detectable anomaly. Thus, we expect the ideal parameters to lie in between these values, and these optimized parameters will be used for the model.

To determine these values, we compute F_1 scores, choose the parameter leading to the highest score and refine the search by selecting a new range around that value to iteratively maximize F_1 . Figure 6 shows these values for the model describing the behavior of *explorer.exe*. The optimal window size for this application was found to be 28.

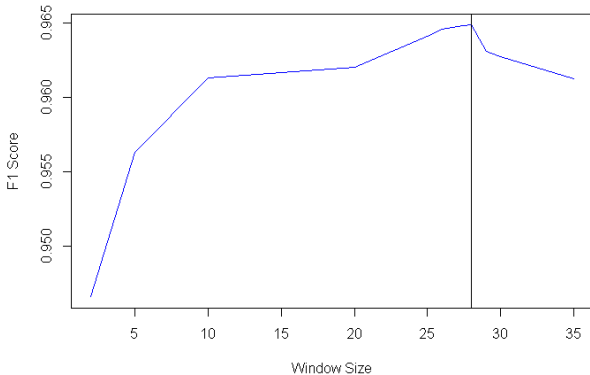


Fig. 6: F_1 score with respect to the window size for *explorer.exe*.

After the optimal parameters are determined, we take the evaluation set and use it to assess the model's performance based on FPR and TPR. We evaluate the model for four different applications from our test set. In addition to a long

living application like *explorer.exe*, we examine the performance on several short to medium-lived applications to see whether they could disguise as each other. Table III shows the resulting values. The first column states the application name, followed by the two parameters, *window* and *threshold factor*. The last column denotes the respective F_1 value. It can be noted that maximizing the F_1 values allows to find a parameter configuration leading to a FPR of zero.

	window	factor	FPR	TPR	F_1
explorer.exe	28	1	0.14	0.9763	0.9706
explorer.exe	28	2	0.0	0.9746	0.9820
ssh.exe	2	2	0.0	0.9910	0.9932
mintty.exe	20	1	0.09	1	0.8789
mintty.exe	20	2	0.0	0.9929	0.9966
texmaker.exe	30	1	0.62	0.9966	0.9930
texmaker.exe	30	2	0.0	0.9913	0.9960

TABLE III: Application models with parameters and respective FPR and TPR.

The fluctuation in FPR for small parameter changes suggests that it is crucial to select their adequate values during the learning phase. In the case of *texmaker.exe* the change from a threshold factor of one to two means a difference of 64% in false positive recognition. The same observation with a less drastic impact can be made for the other applications. The *ssh.exe* showed the same FPR for all configurations close to the one shown in Table III. This means that its behavior is consistent enough to generate a comprehensive model that does not label normal instances as anomalies. These results overall substantiate our hypothesis and suggest that it is possible to distinguish processes by virtual memory fingerprint. Thus, more research on behavioral process profiling can be used to provide a novel direction to improve current state of the art Intrusion Detection Systems.

V. APPLICATION DISCUSSION

The real-time aspect plays an important role for today's IDSs. A good example for this is the hack of the German parliament [24] where information was still being exfiltrated even two weeks after it the breach was has been noticed. This was primarily due to a lack of situational awareness which could have been mitigated using an IDS for anomaly detection.

However, this requirement also introduces new architectural challenges on scalability, processing power and data availability [25]. We thus propose a client-server architecture which keeps the fingerprint and hence the impact on the client machine's resources to a minimum. It also prevents the clients from counterfeiting their results. In environments with sophisticated malware, active avoidance strategies against such systems have been observed, Borders *et al.* [26].

Figure 7 depicts the high level architecture in which several machines feed their data into a computing cluster. The cluster itself stores the received information in databases. This data can then be used to reconstruct attacks post-mortem as well as training material for administrators in a SOC. The algorithm presented in the previous chapter is deployed on the cluster machines. They maintain a monolithic database which holds the application models for each client. Scalable database solutions implementing the NoSQL approach are capable of processing data much faster than relational database systems [27] and are thus to favor. Their design, which enables deploying clusters

of servers to form one single database, also contributes to the scalability of the system.

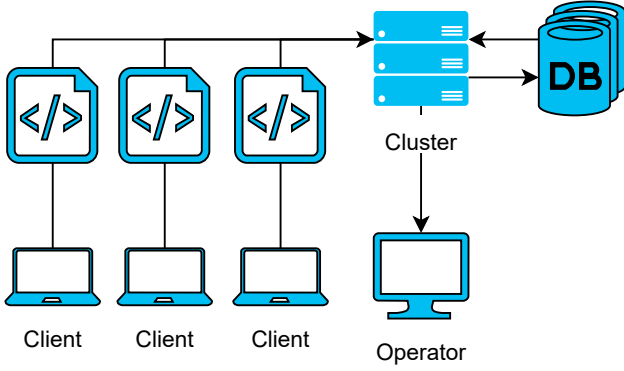


Fig. 7: A client-server architecture to reduce resource consumption on the client machines.

In order to capture the applications straight from the beginning of a system run, the capturing script needs to be installed as a system service. It is necessary to cover the entire uptime of a machine in a real environment as malware otherwise makes use of surveillance gaps to infect the system. Further, for the implementation of the system, we propose a component architecture as illustrated in Figure 8.

The actual system comprises of three different tiers: The two components of the client-server architecture and a middleware serving as a broker between the clients and possibly multiple servers. We suggest ZeroMQ [28] as a message-oriented transport. The advantage of message passing in general is that the client does not have to take care of the message once it has been delivered to the broker. Changes to the communication topology like load-balancing or other dynamic scheduling [29] are thus completely transparent for the client.

On the component level, the client tier consists of two components. The ReadingSource which is responsible for capturing the relevant information from the local machine and passing it on to the QProducer. The actual implementation used for the producer depends on the deployed message middleware. However, regardless of which transport is used, all producers provide the same interface towards the reading source. The counterpart of the producer is the QConsumer. Both components need to implement the same communication protocol in order to share a message broker. They form the bridge between client and server. On the server side the queue consumer writes the incoming messages into the database so that they can

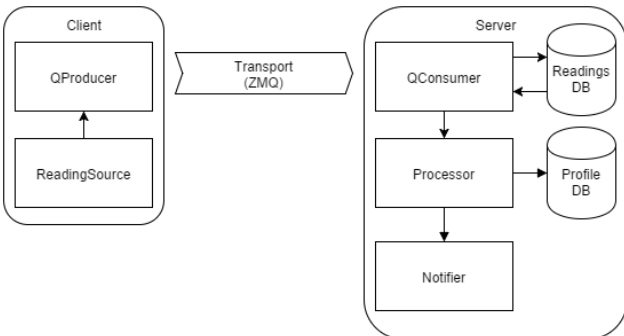


Fig. 8: Data flow among components of the IDS.

be replayed at a later point in time for analysis or training purposes. Furthermore, the consumer forwards all incoming readings to the processor. This component is responsible for creating the application models and detecting anomalies. The models are then stored for each client and incoming readings are checked against the model as soon as the learning phase is completed. The last component on the server is the notifier. It maintains the counter as mentioned in Section III-A. While the counter is constantly decreased over time the notifier triggers an alarm once a threshold is exceeded.

VI. FUTURE WORK

Although, the models might be more stable when including additional features for each process, e.g. CPU utilization or I/O operations. In this paper we gathered virtual memory consumption on a single system over the course of several days. The acquired data was then used to create a model for the respective application to establish a ground truth for anomaly detection. The test data was partitioned into a training and an evaluation set and optimal model parameters were obtained by iterating over their 2D candidate space to optimize the F_1 score of the model. In our case, this lead to models with very small false positive and good true positive rates. However, we also observed that choosing the appropriate parameters is crucial to have a working model since small adjustments can cause a model's FPR and TPR to vary quite a deal.

Incorporating approaches like parallelized machine learning [30] could increase efficiency during the learning phase and thus make the application models available more promptly. Additionally, using malware samples gathered from honey pots, the system could create models for the behavior of those malicious programs. These precomputed models can be provided to other machines to serve as an early knowledge base to bootstrap the system. Although this approach might have drawbacks related to real-time monitoring, it decreases the risk of a malware infection during the system's initial model generation phase. Finally, a method developed by Breunig *et al.* [31] might be worth exploring. Instead of having a binary decision whether a value represents an outlier and thus an anomaly, they compute a local outlier factor which is a continuous measure for how isolated a reading is. A threshold on this measure could exhibit even better performance to detect anomalous behavior. Moreover, the near linear performance of it seems promising for real-time monitoring.

Concluding, for our approach, we implemented and evaluated a novel method for anomaly detection. The reported results show good performance of our system which also allows for real-time monitoring. In addition to the algorithm itself we also discussed an architecture that enables deployment at scale with built-in expansion points. The centralized character of the system takes the responsibility away from each client and enables a security operations center to have a comprehensive view on an organization's IT infrastructure.

REFERENCES

- [1] Cve-2015-5123. [Online]. Available: <https://www.cvedetails.com/cve/CVE-2015-5123/>
- [2] Cve-2015-2638. [Online]. Available: <https://www.cvedetails.com/cve/CVE-2015-2638/>
- [3] Mitm pe injector. [Online]. Available: <https://peinjector.eu/>

- [4] A. Barth, C. Jackson, C. Reis, T. Team *et al.*, "The security architecture of the chromium browser," 2008.
- [5] M. F. Zolkipli and A. Jantan, "A framework for malware detection using combination technique and signature generation," in *Computer Research and Development, 2010 Second International Conference on*. IEEE, 2010, pp. 196–199.
- [6] M. Frank, R. Biedert, E.-D. Ma, I. Martinovic, and D. Song, "Touchalytics: On the applicability of touchscreen input as a behavioral biometric for continuous authentication," *Information Forensics and Security, IEEE Transactions on*, vol. 8, no. 1, pp. 136–148, 2013.
- [7] K. Niinuma, U. Park, and A. K. Jain, "Soft biometric traits for continuous user authentication," *Information Forensics and Security, IEEE Transactions on*, vol. 5, no. 4, pp. 771–780, 2010.
- [8] J. Liu, F. R. Yu, C.-H. Lung, and H. Tang, "Optimal combined intrusion detection and biometric-based continuous authentication in high security mobile ad hoc networks," *Wireless Communications, IEEE Transactions on*, vol. 8, no. 2, pp. 806–815, 2009.
- [9] P. Legg, N. Moffat, J. R. Nurse, J. Happa, I. Agraftotis, M. Goldsmith, and S. Creese, "Towards a conceptual model and reasoning structure for insider threat detection," *Journal of Wireless Mobile Networks, Ubiquitous Computing, and Dependable Applications*, vol. 4, no. 4, pp. 20–37, 2013.
- [10] A. Liu, C. Martin, T. Hetherington, and S. Matzner, "A comparison of system call feature representations for insider threat detection," in *Information Assurance Workshop, 2005. IAW'05. Proceedings from the Sixth Annual IEEE SMC*. IEEE, 2005, pp. 340–347.
- [11] Dealing with svchost.exe virus' sneak attack. [Online]. Available: <http://usa.kaspersky.com/internet-security-center/threats/svchost-exe-virus-attack>
- [12] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM computing surveys (CSUR)*, vol. 41, no. 3, p. 15, 2009.
- [13] T. Lane and C. E. Brodley, "Temporal sequence learning and data reduction for anomaly detection," *ACM Transactions on Information and System Security (TISSEC)*, vol. 2, no. 3, pp. 295–331, 1999.
- [14] R. Sekar, M. Bendre, D. Dhurjati, and P. Bollineni, "A fast automaton-based method for detecting anomalous program behaviors," in *Security and Privacy, 2001. S&P 2001. Proceedings. 2001 IEEE Symposium on*. IEEE, 2001, pp. 144–155.
- [15] Y. Okazaki, I. Sato, and S. Goto, "A new intrusion detection method based on process profiling," in *Applications and the Internet, 2002.(SAINT 2002). Proceedings. 2002 Symposium on*. IEEE, 2002, pp. 82–90.
- [16] S. A. Hofmeyr, S. Forrest, and A. Somayaji, "Intrusion detection using sequences of system calls," *Journal of computer security*, vol. 6, no. 3, pp. 151–180, 1998.
- [17] S. Forrest, S. Hofmeyr, A. Somayaji, T. Longstaff *et al.*, "A sense of self for unix processes," in *Security and Privacy, 1996. Proceedings., 1996 IEEE Symposium on*. IEEE, 1996, pp. 120–128.
- [18] J. Ryan, M.-J. Lin, and R. Miikkulainen, "Intrusion detection with neural networks," *Advances in neural information processing systems*, pp. 943–949, 1998.
- [19] A cross-platform process and system utilities module for python. [Online]. Available: <https://github.com/giampaolo/psutil>
- [20] D. Endler, "Intrusion detection. applying machine learning to solaris audit data," in *Computer Security Applications Conference, 1998. Proceedings. 14th Annual*. IEEE, 1998, pp. 268–279.
- [21] L. Van der Maaten and G. Hinton, "Visualizing data using t-sne," *Journal of Machine Learning Research*, vol. 9, no. 2579–2605, p. 85, 2008.
- [22] P. Zhang, "Model selection via multifold cross validation," *The Annals of Statistics*, pp. 299–313, 1993.
- [23] R. Kohavi *et al.*, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in *Ijcai*, vol. 14, no. 2, 1995, pp. 1137–1145.
- [24] German parliament struggles to purge hackers from computer network. [Online]. Available: <http://www.wsj.com/articles/german-parliament-struggles-to-purge-hackers-from-computer-network-1434127532>
- [25] J. S. Balasubramanian, J. O. Garcia-Fernandez, D. Isacoff, E. Spafford, and D. Zamboni, "An architecture for intrusion detection using autonomous agents," in *Computer Security Applications Conference, 1998. Proceedings. 14th Annual*. IEEE, 1998, pp. 13–24.
- [26] K. Borders, X. Zhao, and A. Prakash, "Siren: Catching evasive malware," in *Security and Privacy, 2006 IEEE Symposium on*. IEEE, 2006, pp. 6–pp.
- [27] N. Leavitt, "Will nosql databases live up to their promise?" *Computer*, vol. 43, no. 2, pp. 12–14, 2010.
- [28] P. Arce, C. Maureira, R. Bonvallet, and C. Fernandez, "Forecasting high frequency financial time series using parallel ffn with cuda and zeromq," in *Information and Telecommunication Technologies (APSITT), 2012 9th Asia-Pacific Symposium on*. IEEE, 2012, pp. 1–5.
- [29] B. A. Shirazi, K. M. Kavi, and A. R. Hurson, *Scheduling and load balancing in parallel and distributed systems*. IEEE Computer Society Press, 1995.
- [30] Y. Low, J. E. Gonzalez, A. Kyröla, D. Bickson, C. E. Guestrin, and J. Hellerstein, "Graphlab: A new framework for parallel machine learning," *arXiv preprint arXiv:1408.2041*, 2014.
- [31] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "Lof: identifying density-based local outliers," in *ACM sigmod record*, vol. 29, no. 2. ACM, 2000, pp. 93–104.

APPENDIX

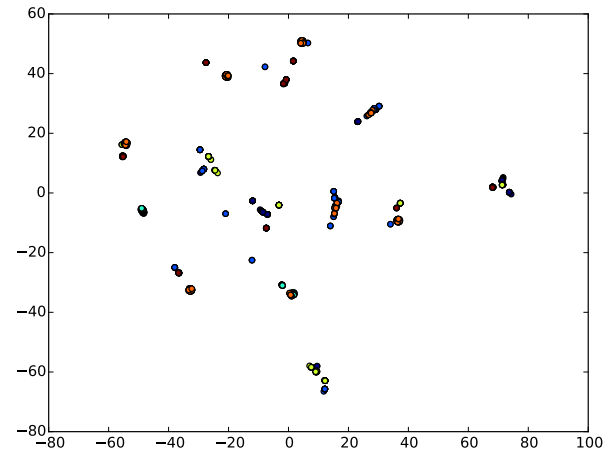


Fig. 9: Features of six processes at windowed with a size of two. Each color represents an individual application.

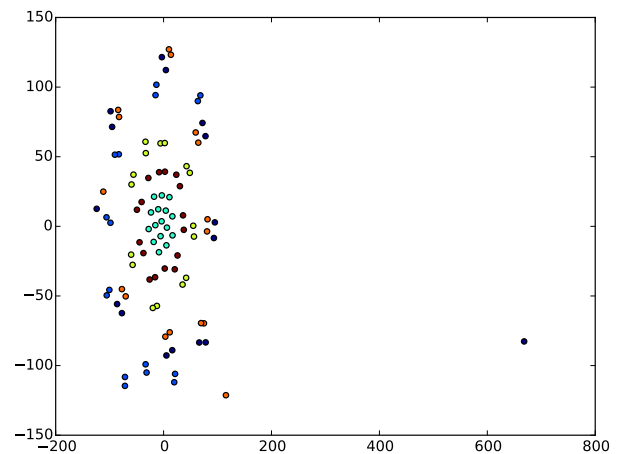


Fig. 10: Features of six processes at windowed with a size of 100. Each color represents an individual application.